

SURVIVAL SCHOOL

An MCQ-driven learning & competitive exam platform for the university

Formal Introduction · Technical Deep Dive · Live Demonstration

Team Matrix Igniters · Sai University · MANTRA 2026

Presenter 1	Formal Introduction & Vision	J.V.A Siva Rama Teja
Presenter 2	Technical Architecture & Engineering	B. Chandra Sekhar Reddy
Presenter 3	Live Product Demonstration	L. Charan Teja

Prepared for a presentation to the Dean and the student body. Every feature, number, and technical claim in this pack reflects the current, working codebase — nothing here is invented or aspirational.

Running Order

A three-person, ~12-minute presentation: one formal introduction, one technical walkthrough, one live demonstration — then open floor for the Dean's questions.

#	Segment	Presenter	Focus	Time
1	Formal Introduction	J.V.A Siva Rama Teja	Problem, vision, who this serves	~3 min
2	Technical Deep Dive	B. Chandra Sekhar Reddy	Architecture, security, engineering rigor	~4 min
3	Live Demonstration	L. Charan Teja	The product, running, in real time	~4 min
4	Q&A	All three	Dean and student questions	~5 min

Stage cue for everyone: speak to the Dean, but glance to the students often — this was built for them as much as for the institution.

PART 1 Formal Introduction

J.V.A Siva Rama Teja · ~3 minutes · Problem, vision, audience

Good afternoon, Dean, faculty, and everyone here today. Thank you for the time to show you something we've been building: **Survival School**.

Every university runs the same three systems side by side, and they rarely talk to each other: a place to teach, a place to test, and a place to keep students motivated enough to keep showing up. Instructors write a quiz in one tool, run a timed exam in another, and have no real way to stop a student from opening a second tab during that exam. Engagement is an afterthought — a leaderboard bolted onto a spreadsheet, if it exists at all.

Survival School is our answer to that: **one platform** that carries a course from its lessons all the way through timed exams, live competitive rounds, certificates, and a campus timetable — with grading the server decides, never the student's browser, and with academic integrity treated as a first-class feature rather than an afterthought.

Concretely, that means: courses with lessons and progress tracking; quizzes and timed exams; a Google-Classroom-style workflow where a lecturer builds a class, shares a join code, and reviews results with one CSV export; live, multiplayer **Elimination Battles** — knockout-style competitive rounds with real-time chat and a room code or QR to join; platform-wide weekly and monthly **contests** with a live leaderboard and real certificates; an AI tutor and AI-generated practice questions for self-study; a full gamification layer — points, streaks, badges; and a campus timetable with clash detection and calendar export.

This isn't a mockup we're describing to you today. It is a working system, already deployed — backend on Render, frontend on Vercel, database on Supabase — with real students able to log in, right now, with a password, a passkey, or two-factor authentication.

In a moment, my colleague will walk you through how it's actually built and why we trust it with something as sensitive as a graded exam. And after that, we'll stop talking about it and simply show it to you, live.

Speaker cues

- *Open with warmth, not a script-read — make eye contact with the Dean before the first line.*
- *Pause after naming the three disconnected systems; let the problem land before the solution.*
- *When you say “already deployed,” gesture toward the laptop/screen that L. Charan Teja will use next.*

PART 2 Technical Deep Dive

B. Chandra Sekhar Reddy · ~4 minutes · Architecture, security, rigor

Thank you. I'll spend a few minutes on how this is actually built, because for a system that grades exams and issues certificates, the engineering has to be trustworthy, not just functional.

The stack. The backend is FastAPI on Python 3.11, fully asynchronous, with SQLAlchemy 2.0 over PostgreSQL as the single source of truth, and Redis for rate-limiting and caching. The frontend is Next.js 15 with React 19 and TypeScript, styled with Tailwind, and it ships as a real installable app — a service worker for offline access, and web push notifications with no third-party account required. In production, the backend runs on Render, the frontend auto-deploys on Vercel, and the database is a real Supabase Postgres instance.

The one rule that matters most: the server is the only authority on correctness. When a student submits an answer, the client sends only their raw selection — never a score, never a 'correct' flag. A single scoring service decides every grade, every time, for quizzes, exams, contests, and battles alike. That single rule is what makes points, badges, leaderboards, and certificates trustworthy in the first place.

Exam integrity is layered, not theatrical. Standard course exams log tab-switches, copy/paste, and fullscreen exits for instructor review — deliberately non-punitive, because a stray notification shouldn't fail a real student. Competitive formats — contests, Elimination Battles, and classroom exams — go further: mandatory fullscreen, tab-switch detection, an opt-in client-side webcam face-presence check that never sends video anywhere, only a violation event, and — as of this week's build — idle detection, so simply walking away from a live exam is caught too. All of it shares one warning system: two strikes, then termination, consistently across every exam type.

Security in depth. Argon2id password hashing with a timing-safe dummy-hash path so an attacker can't tell a wrong password from a nonexistent account. TOTP two-factor authentication and passwordless WebAuthn passkey login. Row-level security policies on every table, because our database is reached directly rather than behind an API gateway. Rate limiting, GDPR data export and deletion, and scheduled dependency scanning with Dependabot on top of it.

We test what we claim. Just under 200 automated backend tests run against a real PostgreSQL and Redis — not mocks — including a genuine concurrency test that fires eight simultaneous exam submissions and proves points are awarded exactly once. A real load test measured roughly a 4.4x throughput improvement from our caching layer. An automated WCAG 2.1 AA accessibility audit ran against every route in both light and dark theme and every violation it found was fixed.

And we're honest about what's still open. Our real-time chat currently uses in-memory connections, so it doesn't yet fan out across multiple backend replicas — documented, not hidden, and on our roadmap. That's the standard we hold ourselves to: say what's true, including the parts that aren't finished yet.

With that foundation, let's stop talking about the architecture — and show you the product itself.

Speaker cues

- *Slow down on “the server is the only authority” — this is the sentence a Dean remembers.*
- *If asked “what happens if a student's laptop lies about the score,” you already have the answer: it can't; the server ignores it.*
- *Say the open limitation (chat, multi-replica) plainly and move on — don't over-apologize for it.*

PART 3 Live Demonstration

L. Charan Teja · ~4 minutes · The product, running live

Thank you. Everything you've just heard, we're now going to show you happening in real time — this is the live, deployed application, not a recording.

I'll move through it quickly: signing in, a student's dashboard, taking a timed assessment with the integrity checks actually engaging, a live competitive round, and a certificate a student can walk away with — one that a future employer could verify without ever calling us.

If anything hiccups — a slow network is the worst enemy of a live demo — I'll narrate what should be happening and we'll come back to it after Q&A.; That's the honest way to run a live demo, and it's exactly what we'd expect from anyone showing us their work.

Demo Runbook

Rehearse this end-to-end at least once on the actual demo network before presenting. Keep a second device or a pre-recorded 90-second backup clip of the Elimination Battle round in case live multiplayer isn't practical in the room.

Step	Screen / Action	Say this
1	Login page — sign in with a passkey (or password)	“No typed password needed here — this is a real WebAuthn passkey.” “No typed password needed here — this is a real WebAuthn passkey.”
2	Student dashboard — points, streak, daily challenge	“Everything here is all computed server-side from real attempt history.”
3	Open a Classroom exam or Practice session; show	“Watch server-side integrity checks actually engaged before the first question loads.”
4	Join or show an Elimination Battle room (room code)	“This is a live multiplayer knockout round — the questions arrive over a websocket.”
5	Submit an answer; show the instant, server-graded result	“The client never told the server if that was right — it just sent the selection.”
6	Open a certificate view page and its “cryptographic proof”	“This certificate is signed — anyone can verify it's genuine without trusting our server.”
7	Instructor/admin analytics view with CSV export	“Instructors see real pass rates and per-question difficulty, not guesses.”
8	Toggle light/dark theme	“One switch, every page — no flash of the wrong theme on load.”

Closing line (all three presenters): “That's Survival School — built, tested, and already running. We'd welcome your questions.”

Appendix A — Feature Reference

A grouped reference of what the platform actually does today, for use while answering questions.

Learning & Assessment

Feature	What it actually does
Courses & lessons	Sections/lessons with per-student progress tracking.
Quizzes & timed exams	Server-authoritative grading; client can never submit a score.
Classroom system	Google-Classroom-style: join codes, hand-picked question sets, scheduled open/close windows, per-student progress tracking.
Practice mode	Practice from bookmarks or a student's own past mistakes; deliberately awards zero points to prevent farming.
Course discussions	Threaded per-course Q&A with instructor-flagged answers.

Competitive & Gamified

Feature	What it actually does
Elimination Battles	Live, multiplayer knockout rounds; room code/QR join; live chat; AI-generated question pools on first use only.
Contests	Auto-scheduled weekly/monthly, platform-wide, live leaderboard, top-3 certificates.
Points, streaks, badges	Full gamification layer with a global leaderboard.
Certificates	Server-computed grade/score, QR-verifiable public page, server-rendered PDF, and Ed25519 cryptographic signature.

Integrity & Trust

Feature	What it actually does
Standard exam monitoring	Tab-blur/copy/paste/fullscreen-exit logged for instructor review — never auto-fails a student.
Competitive-format lockdown	Mandatory fullscreen, tab-switch detection, idle detection, optional client-side face-presence proctoring — never auto-fails a student.
Face proctoring	Runs entirely in-browser (TinyFaceDetector); no video or images ever leave the device.

AI & Engagement

Feature	What it actually does
AI tutor	Pluggable AI provider (Sarvam AI) for conversational help.

AI-generated practice	Structurally isolated from the real question bank; zero gamification points.
Daily challenge & streaks	One shared question per day, wired into the streak system.
Web push & offline	Real Web Push (VAPID) notifications and an installable, offline-capable app shell.

Operations & Admin

Feature	What it actually does
Timetable	Per-course + campus-wide schedules, clash detection, personal CSV/XLSX upload, live “happening now”
Attendance	Short-lived check-in codes tied to the real timetable.
Instructor analytics	Live pass-rate and per-question difficulty, CSV export.
Admin console	User management, audit logs, exam scheduler, instructor applications, campus timetable.
Global search	Public search across published courses and discussions.

Appendix B — Technology Stack

Layer	Technology
Backend API	FastAPI, Python 3.11 (fully async)
ORM / Migrations	SQLAlchemy 2.0, Alembic
Database	PostgreSQL 16 (Supabase in production)
Cache / Rate limiting	Redis 7
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS
Real-time	Authenticated WebSocket chat + live battle/contest updates
Automation	n8n (event-driven notification workflows)
AI	Sarvam AI via a pluggable provider abstraction (mock provider for CI)
Auth	Argon2id passwords, TOTP 2FA, WebAuthn passkeys, JWT sessions
Proctoring	Client-side TinyFaceDetector (@vladmandic/face-api)
PDF / Certificates	WeasyPrint rendering, Ed25519 signatures (via HKDF over JWT_SECRET)
Deployment	Render (backend), Vercel (frontend), Supabase (Postgres) — live today
Also available	Docker Compose + Kubernetes manifests for self-hosted deployment
CI/CD	GitHub Actions: lint, dependency audit, migrations, backend + frontend tests

Appendix C — Engineering Rigor, By the Numbers

Metric	Evidence
~200 automated backend tests	Run against real PostgreSQL + Redis, not mocks — including a genuine 8-way concurrency test.
44+ table schema	Applied via versioned Alembic migrations, not hand-run SQL.
~4.4x throughput improvement	Measured with a real load test after adding Redis caching + DB indexes.
WCAG 2.1 AA audit	Automated Chromium + axe-core scan across every route, both themes; every violation found was
Zero known dependency vulnerabilities	pip-audit, pip-audit, npm audit all clean; Dependabot + scheduled scans keep it that way.
Verified backup/restore drill	Real pg_dump → restore → row-count match → app boot against the restored database.

Independent peer review	A dedicated four-agent adversarial review caught and fixed a critical GDPR-deletion bug before rel
--------------------------------	--

Appendix D — Anticipated Questions From the Dean

Honest, direct answers — including where we say plainly what isn't finished yet.

Q. How do you stop a student from cheating on a timed exam?

Layered, format-appropriate controls: standard course exams log suspicious activity for instructor review without auto-failing anyone, since a false positive shouldn't punish a real student. Competitive formats — contests, battles, classroom exams — require fullscreen, detect tab-switches and idle time, and can require client-side face-presence checks, with a two-warning system before termination. All of this shares one codepath, so a fix in one place protects every exam type at once.

Q. Can a student fake their score by tampering with the app?

No. The server is the sole authority on correctness — the client only ever sends a raw answer, never a grade. This is enforced by one shared scoring service used by every exam format.

Q. Is this actually running, or is it a prototype?

It's deployed today: backend on Render, frontend on Vercel, database on Supabase. What we demo is the live application, not a mockup.

Q. What happens to student data?

Row-level security is enforced at the database layer on every table. Students can export their own data and request full account deletion under GDPR-style rights, both of which are implemented and tested.

Q. What isn't finished yet?

We say this plainly rather than hide it: our real-time chat currently runs on a single backend instance's memory, so it won't yet fan out across multiple server replicas in a large-scale deployment — documented and on our roadmap. The frontend also doesn't yet have an automated browser test suite, though every build passes linting, type-checking, and a full production build, and key flows have been manually verified live.

Q. How is a certificate trusted outside your own system?

Certificates carry an Ed25519 digital signature over their core facts. A public key lets any third party — an employer, another institution — verify authenticity offline, without ever calling our servers.

Q. What would it take to roll this out campus-wide?

The architecture already supports it: the same codebase runs self-hosted via Docker Compose and Kubernetes manifests, or on the managed Render/Vercel/Supabase stack we're demoing today. Scaling further is a matter of provisioning, not rearchitecting.

End of pack. Rehearse the three speech sections aloud at least twice, and run the full live-demo runbook once on the actual presentation network before the real session.